P1F1 SO:
THE CITADEL SYSTEM

Jon Ballanca y Alejandro Cortés Puig

Grupo 32

1. Índice

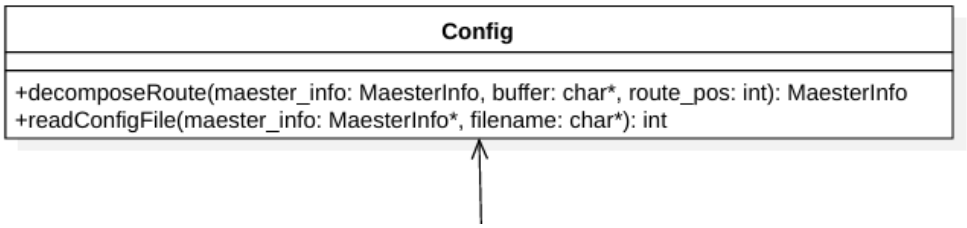
1. Índice 2
2. Diseño: 3
2.1 Diagramas: 3
2.1.1 Config: 3
2.1.2 Inventory: 3
2.1.3 Commands: 4
2.1.4 Maester: 4
2.1.5 Interacción entre modulos 5
2.2 Estructuras de datos: 6
3. Problemas observados y como se han solucionado 9
4. Estimación temporal: 9
5. Conclusiones y propuestas de mejora: 9
6. Bibliografía utilizada: 11

2. Diseño:

2.1 Diagramas:

2.1.1 Config:

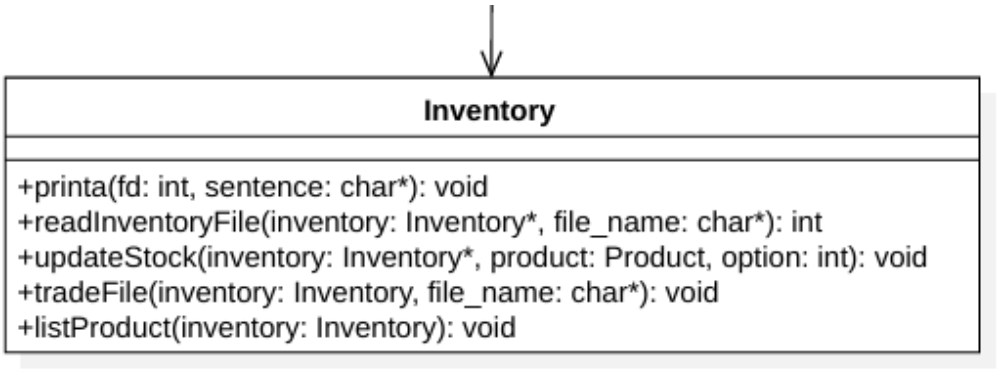
“Config”, es el módulo que se dedica a obtener la información del fichero de comunicación con la finalidad de guardarlo dentro del “maester”.



Las operaciones de este modulo tienen como finalidad guardar e enviar la información del fichero de configuración al “maester”.

2.1.2 Inventory:

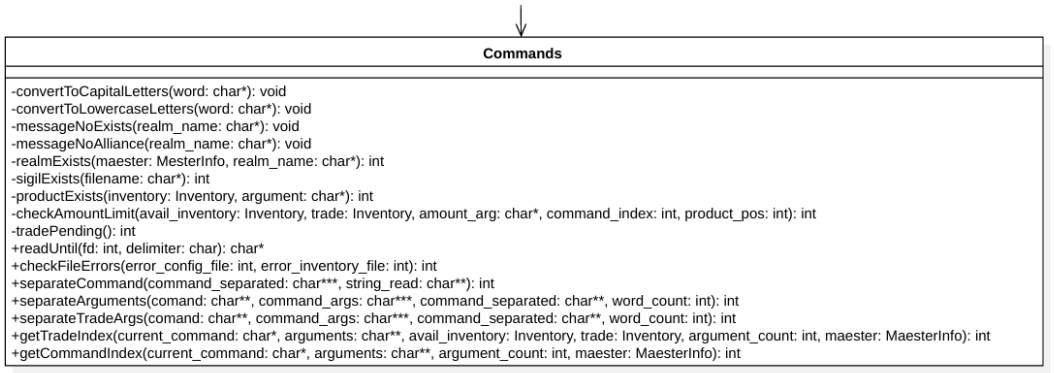
“Inventory” es el módulo encargado de recopilar y gestionar la información que tiene que ver con los productos que tiene cada reino. Por lo que es el responsable de leer los ficheros que almacenan esa información para que puedan ser almacenados dentro del “maester”, actualizar el stock de cada reino en caso de haber recibido un intercambio comercial, crear ficheros para pedir intercambios comerciales y listar los productos disponibles de cada reino si hay una previa alianza establecida.



Las operaciones dentro de Inventory tienen la finalidad de poder gestionar la información de los productos.

2.1.3 Commands:

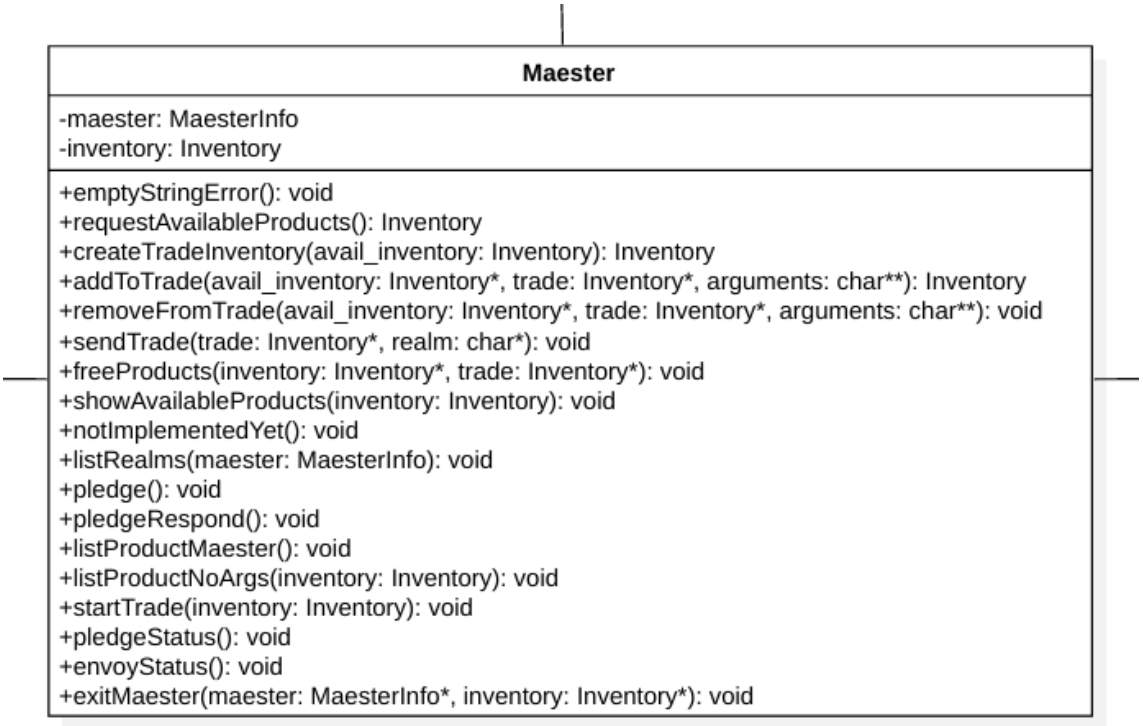
“Commands” es el módulo encargado de la gestión de errores para todos los comandos, se encarga de que tengan la sintaxis correcta, y/o si el comando es posible ejecutarse o requiere de alguna acción previa como una alianza, por ejemplo.



Todas las operaciones de este módulo están diseñadas para la gestión de errores de estos.

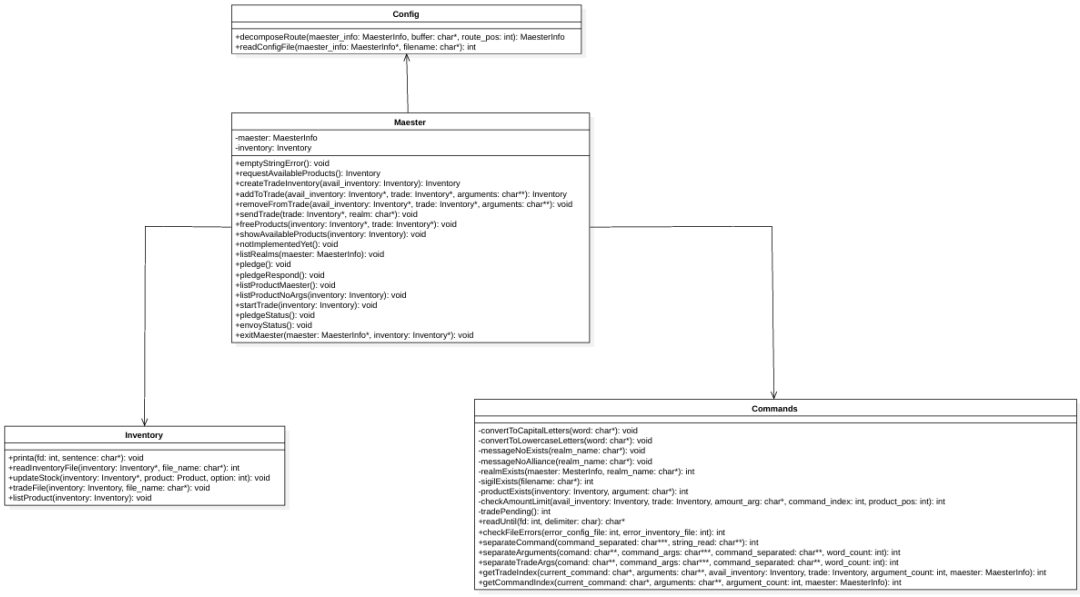
2.1.4 Maester:

“Maester” es el módulo principal, es el encargado de recibir los comandos y decidir que se ha de hacer en cada momento según el comando. Además, es el encargado de almacenar la información del reino. Es el encargado de contactar con los otros módulos en caso de ser necesario.



Todas las operaciones de “Maester” están diseñadas para la gestión de los comandos durante el transcurso del programa.

2.1.5 Interacción entre modulos



Maester es el módulo principal y es el encargado de llamar a los otros módulos en caso necesario.

Al inicio del programa Maester llama a config para obtener la información de su reinado y almacenarla.

Durante el transcurso del programa al recibir un comando, Maester llamará a Commands para saber si hay un error con ese comando o puede proceder con la lógica de este.

Y, para la lógica de algunos comandos se requiere la información de los productos del propio reino o de otro, en ese momento Maester llamará a Inventory para la gestión de esa información.

2.2 Estructuras de datos:

```
typedef struct {
    char name[100];
    int amount;
    float weight;
} Product;

typedef struct {
    Product* products;
    int product_count;
} Inventory;
```

Inventory es la estructura de datos empleada para almacenar los datos dentro del maester, que consiste en:

- Product* producto: Array de productos donde se almacena todos los productos.
- Int producto_count: Cantidad de productos almacenada

A su vez la estructura Producto está definida como:

- Char name[100]: Array de caracteres donde se almacena el nombre del producto.
- Int amount: donde se almacena el precio del producto.
- Float wight: donde se almacena el peso del producto.

```c
// Constantes - Tipo propio de constante (declarado en el .c por limitaciones)
typedef struct {
    const char* name;
    int numArgs;
    int indexConstant;
    char* no_args_error;
} Command;
```

“Command”, es la estructura de datos utilizada para guardar la información específica de cada comando y está definida como:

- Const char* name: Nombre del comando
- Int numArgs: numero de argumentos que se requieren para que el comando se pueda ejecutar.
- Int indexConstant: El índice de comando que es, este número ayuda a la gestión de la lógica de los comandos.
- Char* no_args_error: Indica el mensaje de error específico para cada comando en caso de haberlo.

```c
// Constante - Comandos que existen inicializados como constantes, con su nombre, numero de index, argumento y mensaje de error de argumentos.
const Command COMMAND_CONST[] = {
    {"INVALID COMMAND", 0, 0, ""},
    {"LIST REALMS", 0, 1, "Did you mean to list the realms? Please review syntax."},
    {"PLEDGE", 2, 2, "Did you mean to send a pledge? Please review syntax."},
    {"PLEDGE RESPOND", 2, 3, "Did you mean to send a pledge response? Please review syntax."},
    {"LIST PRODUCTS", 1, 4, "Did you mean to send list all products of a realm? Please review syntax."},          // Sin argumentos
    {"LIST PRODUCTS", 0, 5, ""},
    {"START TRADE", 1, 6, "Did you mean to start a trade with a realm? Please review syntax."},
    {"PLEDGE STATUS", 0, 7, "Did you mean to see a pledge status? Please review syntax."},
    {"ENVOY STATUS", 0, 8, "Did you mean to see an envoy status? Please review syntax."},
    {"EXIT", 0, 9, "Did you mean to exit the system? Please review syntax."}
};

// Constante - Comandos de "trade" que existen inicializados como constantes, con su nombre, numero de index, argumento y mensaje de error de argumentos.
const Command TRADE_COMMANDS[] = {
    {"INVALID COMMAND", 0, 0, ""},
    {"ADD", 2, 1, "Did you want to add a product to the trade? Wrong number of arguments. Please review syntax."},
    {"REMOVE", 2, 2, "Did you want to remove a product from the trade? Wrong number of arguments. Please review syntax."},
    {"SEND", 0, 3, "Did you want to send the trade? Wrong number of arguments. Please review syntax."},
    {"CANCEL", 0, 4, "Did you want to cancel the trade? Wrong number of arguments. Please review syntax."}
};
```

Estos son todos los comandos que se controlan en este programa, con el número de argumentos requerido, el índice utilizado y el mensaje de error específico tanto para los comandos normales, como para los comandos del submenú del comando start trade.

```c
typedef struct {
    char* realm_name;
    struct in_addr ip;
    int port;
    int alliance_exists;
} Route;

typedef struct MaesterInfo{
    char* realm_name;
    char* path;
    int num_envoys;
    struct in_addr ip;
    int port;
    Route* routes;
    int num_routes;
} MaesterInfo;
```

“MaesterInfo” es la estructura de datos que utiliza “config” para poder guardar la información del fichero de comunicación, esta estryctura consta de:

- Char* real_name: Nombre del reino.
- Char* path: Nombre del reino.
- Int num_envoys: numero de “envoys” que tiene el reino.
- Struct in_addr ip: ip del reino.
- Int port: Puerto del reino.
- Route* routes: Array de las rutas que conoce el reino.
- Int num_routes: número de rutas que conoce el reino.

Route se compone de:

- Char* real_name: nombre del reino de la ruta.
- Struct in_addr ip: ip del reino de la ruta.
- Int port: Puerto del reino de la ruta.
- Int Alliance_exists: Entero que indica si hay una alianza con ese reino.

3. Problemas observados y como se han solucionado

Durante el desarrollo de la fase 1 se han encontrado diferentes problemas:

- Problemas con la **lectura de stocks**:

  Se encontraron problemas al leer los ficheros que guardan la información de los "stocks" de los "realms", no leía correctamente.

  Solución: Se revisó el código y parecía correcto por lo que se intuyó que los archivos se podían haber corrompido. Se volvió a descargar los archivos .db y esta vez se leyeron correctamente.

- Problemas con el **control de los comandos que pueden recibir comandos diferentes**, como LIST PRODUCT:

  Durante el protocolo que se ha desarrollado para la gestión de errores, se hacia complicado el gestionar cuando un comando podía tener múltiples posibilidades de argumentos.

  Solución: Se ha considerado como comandos diferentes, por lo que se añadió estos "nuevos" comandos a la estructura de constantes. De esta manera se gestiona el error de manera más eficiente.

4. Estimación temporal:

- Estimación **investigación**: 2 horas.
- Estimación **diseño**: 6 horas
- Estimación **desarrollo del código**: 10 horas
- Estimación de **Testing**: 3 horas
- Estimación **Documentación**: 3 horas.

Total: 24 horas para el desarrollo de la fase 1

5. Conclusiones y propuestas de mejora:

Durante el desarrollo de esta fase, se ha desarrollado un "the cytadel system", un programa con la finalidad de poder forjar alianzas entre reinos y poder comerciar unos con otros. Para ello se ha desarrollado la lógica de todos los comandos necesarios para su correcto funcionamiento, el control de errores de

estos, la lectura del fichero de configuración para obtener la información de los reinos, otra lectura para saber el inventario de cada reino y la creación de un fichero para poder comerciar.

En conclusión, se ha desarrollado el diseño de esta fase pensando en los posibles cambios a futuro, eso implica haberlo hecho más escalable y de una forma más encapsulada.

Propuestas de mejora:

Algunos aspectos de este diseño se pueden mejorar de cara a las siguientes fases.

- Hacer todas las funciones que tengan que ver con el inventario dentro del módulo de “inventory” y no dentro del “maester”.
- El archivo que se crea a la hora de hacer “start trade” tiene un formato que es cómodo para visualizarlo, pero no lo es tanto para obtener información de él.
- A la hora de imprimir por pantalla los nombres de los “realms” después de haber gestionado un comando se muestra en mayúsculas y sería mejor que se mostrara con el nombre original guardado.

6. Bibliografía utilizada:

Salvador Pont, J. (2014). *Programació en UNIX: per a pràctiques de Sistemes Operatius* (Curs 2014–2015). Enginyeria i Arquitectura La Salle.